

## 7[Deadlock Handling]

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint). The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below)

| Process        | Allocation |     |           |     | MAX     |     |           |     |
|----------------|------------|-----|-----------|-----|---------|-----|-----------|-----|
|                | Printer    | ROM | Hard Disk | RAM | Printer | ROM | Hard Disk | RAM |
| Google Drive   | 0          | 0   | 1         | 4   | 0       | 6   | 5         | 6   |
| Firefox        | 0          | 6   | 3         | 2   | 0       | 6   | 5         | 2   |
| Word Processor | 0          | 0   | 1         | 2   | 0       | 0   | 1         | 2   |
| Excel          | 1          | 0   | 0         | 0   | 1       | 7   | 5         | 0   |
| PowerPoint     | 1          | 3   | 5         | 4   | 2       | 3   | 5         | 6   |

| Available Resources |     |           |     |
|---------------------|-----|-----------|-----|
| Printer             | ROM | Hard Disk | RAM |
| 1                   | 6   | 2         | 0   |

Write a program to:

- Calculate current need of resources by each process.
- Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE :

```

GNU nano 8.7 banker.c
#include <stdio.h>

#define P 5 // Number of processes
#define R 4 // Number of resources

int main() {
    int allocation[P][R] = {
        {0,0,1,4}, // Google Drive
        {0,6,3,2}, // Firefox
        {0,0,1,2}, // Word Processor
        {1,0,0,0}, // Excel
        {1,3,5,4} // PowerPoint
    };

    int max[P][R] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[R] = {1,6,2,0};

    int need[P][R];
    int finish[P] = {0};
    int safeSeq[P];
    int work[R];

    printf("\nNeed Matrix:\n");
    // Calculate Need Matrix
    for(int i=0; i<P; i++){
        for(int j=0; j<R; j++){
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    // Copy available to work
    for(int i=0; i<R; i++){
        work[i] = available[i];
    }

    int count = 0;
    while(count < P){
        int found = 0;
        for(int i=0; i<P; i++){
            if(!finish[i]){
                for(int j=0; j<R; j++){
                    if(need[i][j] > work[j])
                        break;
                }
                if(j == R){
                    for(int k=0; k<R; k++){
                        work[k] += allocation[i][k];
                    }
                    finish[i] = 1;
                    count++;
                    found = 1;
                }
            }
        }
        if(!found)
            break;
    }

    printf("\nSafe Sequence: ");
    for(int i=0; i<P; i++){
        if(finish[i])
            printf("%d ", i);
    }
    printf("\n");
}

```

```
GNU nano 2.9.3 banker.c
// Calculate Need Matrix
for(int i=0; i<P; i++){
    for(int j=0; j<R; j++){
        need[i][j] = max{1[j]} - allocation[i][j];
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

// Copy available to work
for(int i=0; i<R; i++){
    work[i] = available[i];
}

int count = 0;

while(count < P){
    int found = 0;

    for(int i=0; i<P; i++){
        if(!finish[i]){
            int j;
            for(j=0; j<R; j++){
                if(need[i][j] > work[j])
                    break;
            }

            if(j == R){
                for(int k=0; k<R; k++){
                    work[k] += allocation[i][k];

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
    }

    if(!found){
        printf("\nSystem is NOT in safe state!\n");
        return 0;
    }
}

printf("\nSystem is in SAFE state.\nSafe Sequence:\n");

char *processName[P] = {
    "GoogleDrive",
    "Firefox",
    "WordProcessor",
    "Excel",
    "PowerPoint"
};

for(int i=0; i<P; i++){
    printf("%s -> ", processName[safeSeq[i]]);
}

printf("END\n");

return 0;
```

Output:

```
Ansh@Anshs-Rog:MSYS ~
$ nano banker.c

Ansh@Anshs-Rog:MSYS ~
$ gcc banker.c -o banker -pthread

Ansh@Anshs-Rog:MSYS ~
$ ./banker

Need Matrix:
0 0 4 2
0 0 2 0
0 0 0 0
0 2 5 0
1 0 0 2

System is in SAFE state.
Safe Sequence:
Firefox -> WordProcessor -> Excel -> PowerPoint -> GoogleDrive -> END

Ansh@Anshs-Rog:MSYS ~
$ |
```